Addestrare GloVe su un corpus italiano

Procedura ex novo con l'implementazione ufficiale Stanford NLP. Dalla preparazione del corpus alla produzione dei vettori parola.

La pipeline è: **corpus tokenizzato / vocabolario / co-occorrenze / shuffle / addestramento**. I pesi vengono inizializzati casualmente: non servono embedding preaddestrati. I comandi seguenti sono pensati per una shell Bash su Linux o macOS.

1. Preparare il corpus

Salva il testo in **corpus_it.txt**, in UTF-8. Ogni riga deve contenere un documento, con i token separati da spazi o tabulazioni. Le nuove righe interrompono il contesto delle co-occorrenze; evita quindi ritorni a capo introdotti soltanto per impaginare il testo. [1]

```
questa è una frase in italiano .
l' elaborazione del linguaggio naturale è interessante .
```

Prima di procedere, elimina markup e contenuti estranei al corpus. Usa una tokenizzazione coerente per apostrofi e punteggiatura, preserva gli accenti e scegli se mantenere le maiuscole. Applica le stesse scelte quando userai gli embedding nel modello finale. Queste sono indicazioni operative; Stanford non impone qui una specifica normalizzazione per l'italiano.

2. Compilare gli strumenti Stanford

Installa Git, GNU Make e GCC o Clang. Clona il repository e compila i programmi C; copia poi il corpus nella directory **GloVe**. Tutti i comandi successivi vanno eseguiti da questa directory. [2]

```
git clone https://github.com/stanfordnlp/GloVe.git
cd GloVe
make

# Copia qui corpus_it.txt prima di proseguire.
```

3. Costruire il vocabolario

Conta le occorrenze dei token e conserva quelli presenti almeno cinque volte. Il file prodotto contiene le parole e le rispettive frequenze. [1, 3]

```
./build/vocab_count -min-count 5 -verbose 2 \
    < corpus_it.txt > vocab_it.txt
```

4. Calcolare e mescolare le co-occorrenze

Costruisci le statistiche con una finestra simmetrica di 15 token per lato. Il parametro **-memory 4.0** indica un budget indicativo di memoria di 4 GB per ciascuna delle due operazioni, eseguite in sequenza. Riducilo se necessario. Lo shuffle prepara le co-occorrenze per l'ottimizzazione. [1, 3]

```
./build/cooccur -memory 4.0 -vocab-file vocab_it.txt \
    -window-size 15 -symmetric 1 -verbose 2 \
    < corpus_it.txt > cooccurrence_it.bin

./build/shuffle -memory 4.0 -verbose 2 \
    < cooccurrence_it.bin > cooccurrence_it.shuf.bin
```

5. Addestrare i vettori

Avvia l'ottimizzazione sulle co-occorrenze mescolate. L'esempio genera embedding di 300 dimensioni, usando 8 thread e 50 passaggi sulle co-occorrenze. [3, 4]

```
./build/glove \
    -input-file cooccurrence_it.shuf.bin \
    -vocab-file vocab_it.txt \
    -save-file vectors_it \
    -vector-size 300 -threads 8 -iter 50 \
    -eta 0.05 -x-max 100 -alpha 0.75 \
    -model 2 -binary 2 -verbose 2
```

Configurazione dell'esempio. Le 300 dimensioni, le 50 iterazioni e la soglia x_max=100 sono valori iniziali da validare sul proprio compito, non una ricetta Stanford specifica per l'italiano. **-min-count** controlla la copertura del vocabolario; **-window-size** l'ampiezza del contesto; **-eta** il tasso iniziale di apprendimento. La disponibilità di RAM e spazio su disco dipende anche dalla varietà delle co-occorrenze.

6. Verificare e utilizzare gli output

vectors_it.txt contiene una parola e 300 valori per riga, senza intestazione con i parametri mostrati. Con **-model 2**, il vettore esportato è la somma del vettore parola e del vettore contesto. **vectors_it.bin** conserva invece i parametri completi in formato binario grazie a **-binary 2**. [4]

Controlla che il training termini senza errori e che i vettori non contengano valori non finiti. Valuta copertura del vocabolario e prestazioni sul compito italiano finale. Per una valutazione corretta, costruisci il corpus di addestramento senza includere i dati riservati al test.

Riferimenti ufficiali

[1] Formato del corpus e strumenti

[2] Repository e compilazione

[3] Pipeline di esempio Stanford

[4] Parametri e formati di output